



Java

Éléments de langage

Ahmad Shahwan

ahmad@digamma.solutions



Du code à l'exécution

Premier programme

```
package com.example;

/**
 * Point d'entrée de l'application.
 */
public class Main {
    /**
     * Méthode principale.
     *
     * @param args non utilisé
     */
    public static void main(String[] args) {
        System.out.println("Hello world");
    }
}
```

- Nom de paquetage `com.example`
- Commentaires JavaDoc
- Nom de la classe `Main`
- Une méthode principale `main()`
- `System` est une autre classe
- `out` est un champ static de la classe `System`
- `println()` est une méthode de la classe de déclaration du champ `out`

Code source : règles de nommage

- Chaque classe doit être sauvegardée dans un fichier à part
 - À l'exception des classes internes
- L'extension du nom de fichier doit être `.java`
- Le nom de fichier doit correspondre au nom de la classe
- Dans notre exemple, le nom de fichier doit être `Main.java`
- Par convention et par souci d'organisation les fichiers source sont organisés dans des dossiers correspondant au paquetages
- Dans notre exemple, le fichier sera placé dans le dossier `com/example`

Compilation du code source

- On utilise `javac` pour compiler le code
 - Avec le nom de fichier en paramètre
- S'il n'y a pas d'erreur de compilation, cela génère un fichier `.class` qui contient le bytecode
 - Il aura le nom de la classe
 - Il sera placé au même niveau que le code source par défaut
 - S'il y a des classes internes, un fichier par classe sera généré
- `javac` fait partie du JDK

```
$ javac com/example/Main.java
```

```
$ ls com/example/  
Main.class Main.java
```

Exécution du bytecode

- On utilise `java` pour exécuter le bytecode généré par la phase de compilation
 - Avec le nom *qualifié* de la classe en paramètre
 - Le nom qualifié : le nom de paquetage plus le nom de classe séparé par un point
- `java` fait partie de la JVM

```
$ java com.example.Main  
Hello world
```

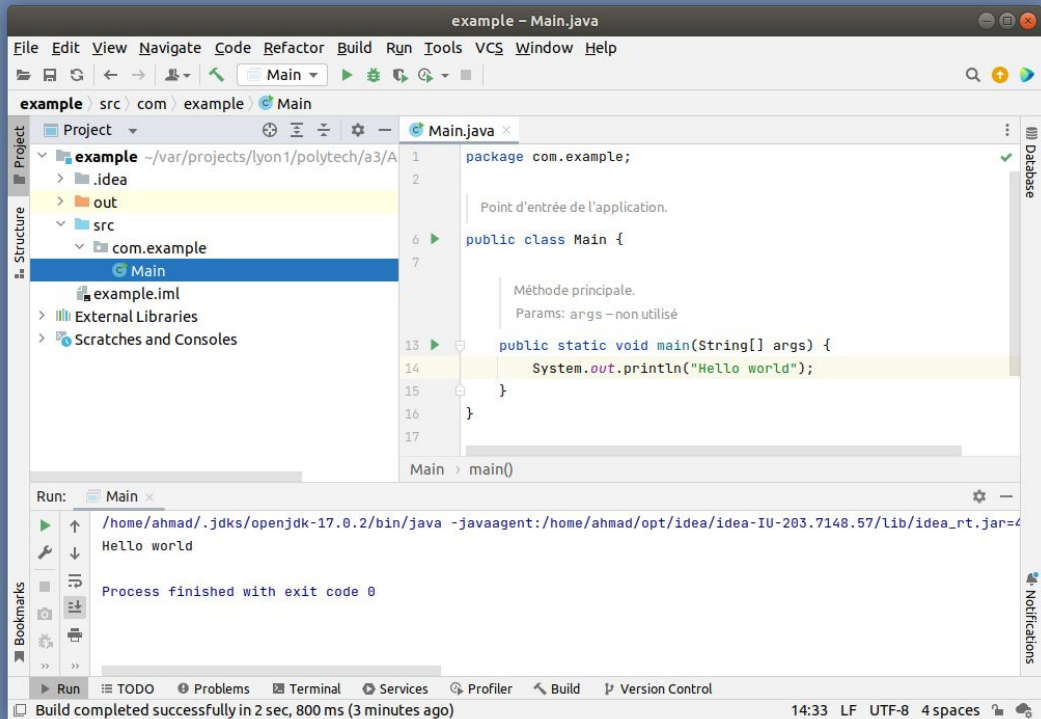
Génération de la documentation

- On utilise `javadoc` pour générer la documentation à partir du code source
 - Avec le nom de fichier en paramètre
- `javadoc` fait partie du JDK

```
$ javadoc com/example/Main.java  
$ open index.html
```



Sinon, utiliser un IDE



Java le langage

Commentaire

Notes sur le code

- Utilisés pour
 - Clarifier le code
 - Documenter le code
- Peut être
 - En bloc ou sur ligne
 - Pour la réalisation ou pour la spécification

Les commentaires

- **Commentaire JavaDoc**

- Utilisés pour documenter la spécification d'une unité de code
- Il ne doit pas se placer à l'intérieur d'une méthode

```
/**  
 * Ceci est un commentaire reconnu par l'outil javadoc.  
 */
```

- **Commentaire de réalisation**

- Commentaires qui terminent à la ligne
- Commentaires qui s'étalent sur plusieurs lignes

```
// Ceci est un commentaire d'une seule ligne  
  
/* Ceci est un commentaire qui commence sur une ligne,  
   et finit sur une autre. */
```

Les identifiants

Règles de nommage

Ce sont les noms qu'on peut
donner au sein d'un programme

Les identifiants

- Utilisés pour donner un nom aux éléments du code
 - Noms des paquetages
 - Noms des classes et interfaces
 - Noms champs et méthodes
 - Nom des variables et paramètres
- Règles de nommage
 - Seulement les caractères alphanumériques et caractère souligné (underscore) sont permis
 - Le caractère dollar (\$) est aussi permis, mais déconseillé car utilisé par le compilateur
 - Une longueur théoriquement illimitée
 - Il est interdit de commencer par un chiffre
 - Un identifiant ne peut être un mot réservé (mot clé ou littéral réservé)

Mots réservés en Java*

- Mots clés

abstract continue fornew switch assert default
goto package synchronized boolean do if private
this break double implements protected throw
byte else import public throws case enum
instanceof return transient catch extends int
short try char final interface static void
class finally long strictfp volatile const float
native super while

- Littéraux réservés

true false null

* À titre informatif

Exemples des identifiants

- `ma-variable` Erreur de compilation
- `1variable` Erreur de compilation
- `ma#variable` Erreur de compilation
- `class` Erreur de compilation (mot clé)
- `MON_CONSTANT` Correct
- `variable1` Correct
- `maVariable` Correct
- `MaClasse` Correct

Les Variables

Une variable est une valeur
nommée

- Une variable peut être :
 - Un champ (static ou non) défini à l'extérieur d'une méthode
 - Une variable locale définie à l'intérieur d'une méthode
 - Un paramètre défini dans l'en-tête d'une méthode
- Le nom de variable
 - Un identifiant
 - Par convention il suit la notion camelCase

Type de variable

- Chaque variable possède un type qui doit être connu avant toute utilisation
 - Typage fort
- Une mauvaise manipulation des types de données génère une erreur de compilation
- Une fois déclaré, le type d'une donnée ne peut pas changer
 - Typage statique

Les champs

- Les champs sont des variables définies à l'extérieur d'une méthode
 - À l'intérieur d'une classe ou interface
- Un type et un niveau de visibilité
- Par défaut, la valeur d'un champ est définie au sein de l'objet (l'instance)
- Un champ peut être statique : valeur définie au sein de la classe
- Un champ peut être final : sa valeur est constante

```
public class CompteBancaire {  
    public final String iban = "FR7630006000011234";  
    private double solde;  
    static int nombreDeComptes = 0;  
}
```

Les champs : déclaration

- Les champs sont déclarés à l'intérieur de la classe
- Syntaxe : *modificateurs type identifiant initialisation ;*
- Seulement type et identifiant sont obligatoire
- Les modificateurs
 - Modificateurs de visibilité : `public`, `private`, `protected`
 - Modificateur pour champs finaux : `final`
 - Modificateur pour champs statiques : `static`
- Si le champ n'est pas initié explicitement, il le sera par défaut
 - À zéro pour les valeurs numérique, à `false` pour le type booléen, à `'\0'` pour le type caractère et à `null` pour les types objet

Les variables locales

- Des variables définies dans une méthode
- Une variable locale peut être *finale* : une fois initiés, sa valeur ne change pas
 - Immutabilité garantie par le compilateur
- Type explicite ou implicite lors d'une initialisation
 - `var` : mot-clé nom de type réservé
- Une variable locale doit être initialisée avant toute autre utilisation
 - Soit dans la déclaration soit plus tard
 - Sous peine d'une erreur de compilation
- Déclaration multiple possible

```
public static void main(String[] args) {  
    // Déclaration sans initialisation  
    int nombreDePiece;  
    // Déclaration avec initialisation  
    String nom = "Jean";  
    // Déclaration d'une variable finale  
    final double longueur = 12;  
    // Déclaration avec type implicite (double)  
    var hauteur = 13.5;  
    // Déclaration multiple, avec et sans init  
    int x, y = 5, z = 6;  
}
```

La portée de variable locale

- Chaque locale a une *portée*, au delà de laquelle elle n'est plus accessible
- La portée est définie par la zone du code entre sa déclaration et la fin du bloc
 - Délimité par une accolade de fermeture

```
{  
    {  
        doSomething();  
        int x;  
        x = 42;  
        use(x);  
    }  
    doAnotherThing();  
}
```

// Hors portée (avant déclaration)
// Déclaration
// Portée de variable x
// Portée de variable x

// Hors portée (hors bloc)

Les paramètres

- Les paramètres sont les entrées d'une méthode
- Accessibles partout dans la méthode
 - La portée s'étale sur tout le corps
- Un paramètre peut être *final*: sa valeur ne peut changer, au risque d'erreur de compilation
- Peuvent être vus comme cas spécial des variables locales
 - Le type doit être explicite

```
public static double calculerSurface(  
    // paramètre final (conseillé)  
    final double hauteur,  
    // paramètre non final  
    double longueur  
) {  
    return hauteur * longueur;  
}
```

Types de Données

Types en Java

Deux catégories

- Types primitifs
 - Les valeurs ne sont pas objets
 - Prédéfini par le langage
 - Stockage mémoire **par valeur**
 - Opérateurs prédéfinies
- Types objet
 - Extensible
 - Le langage fournit certains
 - Stockage mémoire **par référence**
 - Valeur stocké en tas
 - Création par instantiation
 - Avec le mot clé **new***
 - Opérations interdites**

* À l'exception de `String`, `Class` et `Enum`

** À l'exception de `String`

Types primitifs

Type Java	Représentant	Taille en mémoire
byte	entier	1 octet
short	entier	2 octets
int	entier	4 octets
long	entier	8 octets
float	réel	4 octets
double	réel	8 octets
char	caractère ou entier positif	2 octets
boolean	booléen	non défini (1 bit suffit)

Conversion entre type numérique primitifs

- Conversions élargissantes
 - Sans prendre de donnée
 - Peuvent se faire implicitement
 - Entre deux types dans ce sens : byte → short → int → long → float → double
 - De char à int, long, float, ou double de l'autre
- Conversions rétrécissantes
 - Potentielle perte de donnée
 - Nécessite une conversion explicite
 - Entre deux types dans ce sens : double → float → long → int → short → byte
 - Conversion vers char ou char vers byte ou short

Conversion entre type numérique primitifs

```
byte b = 1;
short s = b;
int i = s, i1 = b;
long l = i, l1 = s, l2 = b;
float f = l, f1 = i, f2 = s, f3 = b;
double d = f, d1 = l, d2 = i, d3 = s, d4 = b;
char c = 'A';
i = c; l = c; f = c; d = c;
```

Conversions élargissantes implicites

```
double d = 1D;
float f = (float) d;
long l = (long) f, l1 = (long) d;
int i = (int) l, i1 = (int) f, i2 = (int) d;
short s = (short) i, s1 = (short) l, s2 = (short) f, s3 = (short) d;
byte b = (byte) s, b1 = (byte) i, b2 = (byte) l, b3 = (byte) f, b4 = (byte) d;
char c1 = (char) d, c2 = (char) f, c3 = (char) l, c4 = (char) l, c5 = (char) i, c6 = (char) s;
```

Conversions rétrécissantes explicites

Type objet

- Des classes ou des interfaces
- Les valeurs sont des références à des objets
 - Instances des classes concrètes
- Des types personnalisables
 - On peut en créer autant qu'on veut (nouvelles classes ou nouvelles interfaces)
 - Possiblement à partir d'un type objet qui existe déjà (héritage)

Les classes d'emballage

- Il y a des cas où on a besoin d'utiliser un type primitif par référence : en tant qu'un objet.
- Pour chaque type primitif, il existe une classe qu'on appelle la classe d'emballage ou la classe *wrapper*.

Type primitif	Type d'emballage
<code>byte</code>	<code>Byte</code>
<code>short</code>	<code>Short</code>
<code>int</code>	<code>Integer</code>
<code>long</code>	<code>Long</code>
<code>float</code>	<code>Float</code>
<code>double</code>	<code>Double</code>
<code>char</code>	<code>Character</code>
<code>boolean</code>	<code>Boolean</code>

Les classes d'emballage

- Une valeur de type primitif peut se convertir en classe d'emballage de ce type implicitement.
 - On appelle cette conversion **autoboxing**.
- Une valeur de type classe d'emballage peut se convertir en type primitif implicitement.
 - On appelle cette conversion **unboxing**.

```
// Boxing explicite  
Integer i1 = Integer.valueOf(42);
```

```
// Autoboxing  
Integer i2 = 42;
```

```
// Unboxing explicite  
int i3 = i1.intValue();
```

```
// Unboxing implicite  
int i4 = i1;
```

Valeurs Littérales

Valeurs immédiates

- Chiffres (entiers, réels)
- Valeurs textuelles (caractères et chaînes de caractères)
- Valeurs de vérité
- Valeurs de classes

Valeurs Littérales

- Ce sont des expressions typées
- Ce sont des expressions constantes
- Elles peuvent être utilisées là où une expression est attendue :
 - Lors de l'initialisation d'une variable
 - En tant qu'opérande
 - En tant que paramètre
 - Etc.

```
// valeurs littérales pour initier une variable  
float temperature = 22.5f;  
  
// valeurs littérales en tant qu'opérande  
if (temperature > 12) {  
  
    // valeurs littérales en tant que paramètre  
    System.out.println('C');  
}
```


Valeurs Littérales : entier

- Peuvent être exprimés :
 - En décimal **42**
 - En octal **052**
 - En hexadécimal **0x2A**
 - En binaire **0b101010**
- Pour créer une valeur de type long
 - Ajouter L à la fin
- Un caractère souligné (underscore) peut être ajouté pour améliorer la lisibilité
 - Seulement entre deux chiffre

```
int answer;           // valeurs égales
answer = 42;          // décimal
answer = 052;         // octal
answer = 0x2A;        // hexadécimal
answer = 0b101010;    // binaire

short shortAnswer = 42; // en short lors d'init
long longAnswer = 42L;  // en long avec L

byte maxByte = 127;    // 128 n'aurait pas compilé

int ts = 379_908_000;   // séparateur de milliers
```

Valeurs Littérales : point flottant (réel)

- Peuvent être exprimés :
 - Forme décimale `3.14`
 - Forme scientifique `0.314e1`
- Type par défaut : `double`
- Pour créer une valeur de type `float`
 - Ajouter F (ou f) à la fin
- Un entier peut être forcé en réel
 - En ajoutant D ou F (ou d ou f) à la fin
- Un caractère souligné (underscore) peut être ajouté pour améliorer la lisibilité
 - Seulement entre deux chiffres

```
double phi = 1.6180339; // forme décimale
double c = 2.9979245e8; // notation scientifique

float e = 2.718F;        // valeur float
double montant = 13D;    // valeur double

double pi = 3.141_592_653_589_793_238_46
```

Valeurs Littérales : caractère

- Peuvent être exprimées
 - En caractère entre deux quotes 'A'
 - En chiffre positif 65
- Les caractères spéciaux sont échappés
 - À l'aide de l'antislash (\)
 - '\b' (backspace), '\t' (tab), '\n' (line feed), '\f' (form feed), '\r' (carriage return), '\"' (double quote), '\'' (single quote), '\\' (backslash) et '\0' (null character).
- Une séquence d'échappement unicode peut être utilisée
 - '€' peut être codé '\u20AC'

```
char a;           // valeur égales
a = 'A';          // caractère simple
a = 65;           // chiffre
a = '\u0041';     // séquence unicode
```

```
char newLine = '\n';
char quote = '\\';
```

Valeurs Littérales : booléen

- Uniquement une de deux valeurs littérales
 - true
 - false

```
boolean t = true;  
boolean f = false;
```

Valeurs Littérales : chaîne de caractères

- `String` est une classe prédéfinie en Java (module `java.base`)
- Il représente une chaîne de caractères
- Les valeurs de ce type sont définies entre deux guillemets
 - Exemple `"Bonjour tout le monde"`
- Mêmes séquences d'échappement que pour type caractère

```
String message;  
message = "Hi";  
message = new String(new char[] { 'H', 'i' });
```

Valeurs Littérales : classe

- La classe `Class` est instanciée par la JVM
- Pour accéder à une valeur on utilise les *valeurs de classe*
- `NomDuType.class`
 - Ex. `String.class`
 - Cela renvoie un objet de type `Class` qui représente la classe `String`
- Peut être utilisé pour les types primitifs

```
// Littéral de classe pour classe String  
Class<?> classString = String.class;
```

```
// Littéral de classe pour type primitif int  
Class<?> classInt = int.class;
```

Les tableaux

Valeur multiple du même type

- Un tableau est un objet récipient contenant un nombre fixe des valeurs du même type.
 - La taille d'un tableau est définie lors de la création du tableau
 - La taille ne peut pas changer en suite

Les tableaux : déclaration

- Comme toute variable, les tableaux sont déclarés avec un type et un nom de variable : un identifiant
- Le type de tableau est identifié par une paire de crochets (`[]`) poursuivant le type d'élément
 - La paire de crochets peut poursuivre le nom de variable : syntaxe déconseillé
- La déclaration ne contient pas la taille
 - La taille ne fait pas partie du type de tableau

```
// Déclaration d'un tableau d'entier  
int[] masses;  
  
// Une autre façon, déconseillé  
int aNePasFaire[];  
  
// Déclaration d'un tableau de chaine de caractère  
String[] noms;
```


Les tableaux : création

- Lors de la création d'un tableau, la taille doit être précisée.
- La syntaxe suivant crée un tableau de la taille fournie. Les éléments de tableau prennent des valeurs par défaut
`new type[taille]`
- Un autre syntaxe permet de créer un tableau à partir des valeurs. La taille est omise car déduit par la liste
`new type[] { valeur1, valeur2... }`
- Lors d'une déclaration, le mot clé `new` et le type peuvent être omis

```
// Déclaration et initialisation
int[] masses = new int[3];

// Une nouvelle valeur de type tableau est créée
masses = new int[] { 3_301, 4_867, 5_972 };

// Déclaration et initialisation avec liste
String[] noms = { "Mercure", "Vénus", "Terre" };

// On a le droit à une virgule supplémentaire
double[] densites = { 5.43, 5.24, 5.51, };
```

Les tableaux : accès et manipulation

- On accède à un élément d'un tableau en utilisant un indice entre deux crochets
 - Lecture et écriture
- L'indice est le placement d'élément dans le tableau, en commençant par zéro
 - L'indice 5 indique le sixième élément
- Essayer d'accéder à un élément au delà de la taille du tableau génère une exception
 - Erreur lors d'exécution
- On obtient la taille d'un tableau avec la propriété `length`

```
// Déclaration et initialisation
int[] masses = { 3_300, 4_867, 5_972 };

// Lecture du troisième élément
int masseTerre = masses[2];

// Écriture du premier élément
masses[0] = 3_301;

// La taille du tableau : 3
int taille = masses.length;

// ArrayIndexOutOfBoundsException au runtime
int accesInterdit = masses[3];
```

Les tableaux multidimensionnels

- Le type d'élément de tableau peut, lui aussi, être un tableau
 - On parle d'un tableau multidimensionnel
- Les tailles des sous-tableaux ne sont pas nécessairement égales
 - Les tableaux peuvent être irréguliers
- Seulement la première taille est obligatoire lors de création d'un tableau
- On peut préciser les autres tailles
 - Cela créera les sous-tableaux avec la taille indiquée

```
// Tableau 2D non-régulier
int grille[][] = new int[3][];
grille[0] = new int[2];
grille[1] = new int[] { 11, 13 };
grille[2] = new int[] { 42 };

// Tableau 3D régulier
double matrice[][][] = new double[3][3][4];

// Tableau 2D défini par valeur de ces éléments
String[][] noms = {
    { "Mercure", "Mercury", },
    { "Vénus", "Venus", },
    { "Terre", "Earth", "Erde", },
};

String terreEnAnglais = noms[2][1];
```

Les opérateurs

Les opérateurs

- Un opérateur est un symbole spécial, ou un mot clé, qui manipule un, deux ou trois valeurs et retourne un résultat
 - Les valeurs manipulées sont appelées opérandes dans le contexte d'un opérateur
 - Un opérateur en Java est donc soit unaire, soit binaire, soit ternaire
- Les opérateurs en Java sont définis par le langage et ne peuvent pas être surchargés, modifiés ou augmentés

Opérateurs en Java

Opérateurs	Binaire	Unaire
Arithmétiques	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>	<code>-</code> <code>+</code>
Décalage	<code><<</code> <code>>></code> <code>>>></code>	
Comparaison	<code>==</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code> <code>instanceof</code>	
Bit-à-bit	<code>&</code> <code> </code> <code>^</code>	<code>~</code>
Logiques	<code>&&</code> <code> </code>	<code>!</code>
Affectation	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code> <code>&=</code> <code>^=</code> <code> =</code> <code><<=</code> <code>>>=</code> <code>>>>=</code>	<code>++</code> <code>--</code>
Ternaire	<code>?</code> <code>:</code>	

Opérateurs arithmétiques

- Dans l'ordre de priorité
 - Unaire : + et -
 - Multiplicatives : *, / et %
 - Additives : + et -
- Le type de résultat est
 - double, si un des opérande est double, sinon
 - float, si un des opérande est float, sinon
 - long, si un des opérande est long, sinon
 - int
- Le résultat n'est jamais byte, char ou short

```
int compte = 7;
int memeCompte = +compte;
int compteNegatif = -compte;

int div1 = compte / 2; // quotient entier : 3
double div2 = compte / 2.0; // quotient réel : 3.5
int reste = compte % 2; // reste : 1

short s1 = 5;
int i1 = +s1; // résultat est en int
int i2 = s1 + 1; // résultat est en int
```

Opérateurs de comparaison

- Test d'égalité `==` et `!=`
 - Comparent les valeurs primitifs
 - Comparent les références d'objets
- Test d'inégalité `>`, `<`, `>=`, `<=`
 - Entre valeurs primitifs (ou de leurs classes d'emballage)
- Une opérande peut être promue à
 - `double`, si l'autre opérande est `double`, sinon
 - `float`, si l'autre opérande est `float`, sinon
 - `long`, si l'autre opérande est `long`, sinon
 - `int`

```
// Comparaison entre entiers
boolean cEstLeMeme = (42 == 052);

// Entre réels, attention à la précision
if (42 == (4.2 * 10)) {
    System.out.println("On a eu de la chance");
}

// Comparaison d'identité : c'est false ici
if ("Hi" == new String("Hi")) {
    System.out.println("C'est la fin du monde");
}
```


Opérateurs logiques

- `&&` calcule le ET logique et `||` calcule le OU logique
- S'applique aux opérandes de type booléen
- L'expression de la deuxième opérande peut être court-circuitée :
 - Elle ne sera évalué que si besoin
- `!` calcule la négation logique

```
int reponse = 42;
boolean reponseEstPaire = reponse % 2 == 0; // true

// Attention, i ne sera pas augmenté
if (reponseEstPaire || (++reponse > 0)) {
    faireQuelqueChose();
}

// Négation logique
if (!reponseEstPaire) {
    System.out.println("Réponse est impaire");
}
```

Opérateur de comparaison de type

- `instanceof` détermine si un objet est non nul et appartient à un type donné
- Si le type est une class, l'objet doit être une instance de cette classe, ou d'une de ses sous-classes
- Si le type est une interface, l'objet doit être une instance d'une classe qui réalise cette interface (directement, ou indirectement via une superclasse)
- Quand la première opérande est `null`, l'opérateur renvoie `false`
- Pattern matching depuis Java 16*

```
// false peu importe le type
boolean faux = null instanceof Object;

Object objet = "Salut";

// message est une instance de String
if (objet instanceof String) {
    System.out.println("Vous avez un message.");
}

// Pattern matching en Java 16
if (objet instanceof String message) {
    System.out.println(message);
}
```

* À titre informatif

Opérateurs de décalage et de bit-à-bit

- S'appliquent uniquement sur des entiers
- Se calculent à la base de la représentation binaire des opérandes
 - D'où le nom bit-à-bit (bitwise)
- $\sim$ calcule le complément binaire
 - 0..0 00101010 l'entier 42
 - 1..1 11010101 l'entier ~ 42
- $\&$ calcule ET entre les bit respectifs
 - 0..0 00101010 l'entier 42
 - 0..0 00000011 l'entier 3
 - 0..0 00000010 l'entier $42 \& 3 = 2$
- $|$ calcule OU inclusif, et $\wedge$ le OU exclusif entre les bit respectifs
- Décalage d'un bit: $\ll$ à gauche, $\gg$ à droite en gardant le signe, $\ggg$ à droite

Opérateur ternaire et la concaténation

- `?` : représente un choix entre deux expression, selon une condition
- Seulement une de deux expression sera évaluée
 - L'autre sera court-circuitée
- L'opération `+` peut être utilisée entre deux chaînes caractère
 - Elle sert à les concaténer
- La deuxième opérande peut être de n'importe quel type
 - Sa représentation textuelle sera utilisée

```
int reponse = 42;

// Opération ternair
String parite = (reponse % 2 == 0) ? "paire" :
"impair";

// Opération de concaténation
System.out.println("La réponse " + reponse + " est
" + parite);
```

L'affectation

Affecter une valeur à une variable

- La première (potentiellement la seule) opérande est la variable cible
 - Elle ne doit pas être finale
- La compatibilité de type source et type destination est vérifiée par le compilateur

Affectation

- Une affectation est une expression
 - Une valeur est renvoyée à la fin
- Affectation simple avec la signe =
- Incrémentation par un avec ++
 - Postfix : s'évalue à l'ancienne valeur
 - Prefix : s'évalue à la nouvelle valeur
 - -- pour décrémentation par un
- Affectation combinée à une opération
 - += -= *= /= &= |= ^= %<= >= >>=
 - *Presque* équivalent à une opération, puis affectation
 - Permet de réaliser certain opération sur des type entier autre que `int`

```
// Affectation lors de déclaration
short s = 0;
// Une affectation est une expression
System.out.println(s = 1); // 1
// Incrémentation postfix
s = 0;
System.out.println(s++); // 0
// Incrémentation prefix
s = 0;
System.out.println(++s); // 1
// Décrémentation
s--;
// Incrémentation d'une valeur non 1
s += 5; // s = s + 5 n'aurait pas marché
// Multiplication et affectation
s *= 2; // s = s * 2 n'aurait pas marché
```

Priorité d'évaluation

- Certains opérateurs sont plus prioritaires à d'autre
 - Ça veut dire qu'ils seront évalués d'abord, si l'expression ne précise pas d'ordre
- Le tableau suivant liste les opérateurs dans l'ordre descendante de priorité*
- Pour modifier l'ordre d'évaluation, il faut regrouper les opérateurs entre parenthèses
- Par souci de clarté, il est recommandé de toujours regrouper les opérateurs

Unaire	<code>++ -- + - ~ !</code>
Arithmétique	<code>* / % + -</code>
Décalage	<code><< >> >>></code>
Comparaison	<code>< > <= >= instanceof == !=</code>
Bit-à-bit	<code>& ^ </code>
Logique	<code>&& </code>
Ternaire	<code>? :</code>
Affectations	<code>= += -= *= /= %= &= ^= = <<= >>= >>>=</code>

* À titre informatif

Les Méthodes

Unité d'exécution

- Une opération nommée qui représente une fonctionnalité cohérente

Les méthodes

- Une méthode peut être
 - Concrète : accompagnée d'une réalisation
 - Abstraite : sans réalisation
- Une méthode d'objet ou une méthode de classe
 - Les méthode de classe sont appelées méthode statiques en Java
- Une méthode prend une liste ordonnée, possiblement vide, de paramètres
 - Les paramètres sont typés
- Une méthode peut avoir un type de retour
- Une méthode peut avoir une liste des erreurs qui peuvent être levées par sa réalisation

Les méthodes : le syntax

- Syntaxe de déclaration d'une méthode
modificateurs **type_de_retour nom** (list_des_parametres) clause_throws
code_ou_point_virgule
- Les modificateurs, la liste des paramètres et la clause `throws` sont optionnel

Modificateur des méthodes

- `public`, `private`, `protected` : visibilités
- `final` : signifie que la méthode ne peut pas être redéfinie dans une sous-classe
- `abstract` : signifie que la méthode n'a pas de réalisation
- `synchronized` : seulement un thread à la fois a le droit d'exécuter la méthode
- `static` : signifie que la méthode s'exécute dans le contexte de la classe, et non pas l'objet

```
public final String getRib() {  
    ...  
}  
  
public synchronized boolean debiter(double m) {  
    ...  
}  
  
protected abstract double calculerInteret();  
  
public static boolean verifierRib(String rib) {  
    ...  
}
```

En-tête de la méthode

- Type de retour : peut être n'importe quel type valable en Java (primitif ou référence)
 - Lorsque la méthode ne renvoie pas de valeur de retour, le pseudo-type `void` doit être utilisé
- La liste des paramètres:
 - Une liste ordonnée des paramètres typés, séparés par des virgules
- Clause `throws`
 - Identifie les exceptions que la méthode peut lever
 - Le mot clé `throws` est suivi d'une liste de type d'exception, séparés par des virgules
 - Les types sont des sous-classes de `Throwable`

Corps de la méthode

- Les méthodes concrètes doivent avoir un corps : un bloc du code qui suit l'en-tête de la méthode
 - Sinon, l'en-tête est terminé par point-virgule
- Le mot clé `return` sert à renvoyer une valeur et sortir de la méthode
 - Si la méthode ne renvoie pas de valeur `return;` sort de la méthode simplement
- Le mot clé `throw` sert à lever une exception : un objet de type `Throwable`
- Si la méthode a un type de retour différent de `void`, le compilateur vérifie que tous les chemins d'exécution mènent soit à `return` soit à un `throw`

Les méthodes : exemple

```
public synchronized String faireVirement(String ribDestinaire, double montant)
    throws CompteNonValableException, CompteNonTrouvableException, SoldeNotSuffisanteException {
    if (!CompteBancaire.verifierRib(ribDestinaire)) {
        throw new CompteNonValableException();
    }
    CompteBancaire destinataire = CompteBancaire.trouverParRib(ribDestinaire);
    if (!this.debiter(montant)) {
        throw new SoldeNotSuffisanteException();
    }
    destinataire.crediter(montant);
    String[] transitions = this.getTransitions();
    if (transitions.length != 0) {
        return transitions[transitions.length - 1];
    }
    return "";
}
```

Signature de méthode

- La signature d'une méthode est la combinaison de son **nom** et la **liste triée des types de ses paramètres**
- On peut avoir deux méthodes qui partagent le même nom au sein de la même classe, tant qu'elles ont deux signatures différentes
 - On parle de la **surcharge** d'une méthode
- Ni le type de retour ni la liste d'exceptions fait partie de la signature
 - On ne peut pas avoir deux méthodes dans la même classe qui se distinguent uniquement par le type de retour ou la liste d'exceptions

Méthode variadiques

- Une méthode variadique accepte un nombre indéfini d'argument
- Elle est identifiée par un **dernier** paramètre avec trois point (. . .) après le type
- Ce paramètre est traité en tant que tableau à l'intérieur de la méthode
- Pour les appelants, un nombre varié (zéro ou plus) de paramètres du même type peut être passé

```
/**
 * Méthode System.out.printf()
 */
public PrintStream printf(
    String format,
    Object... args) {
    return format(format, args);
}

...

System.out.printf(
    "Valeur de Pi : %f. Valeur de e : %f",
    Math.PI, Math.E);
```


Structure de contrôle

Structure conditionnelle `if`

- Dans sa forme simple, conditionne l'exécution d'une partie du code
 - Mot clé `if` suivi d'une condition entre deux parenthèses
 - Le tout suivi d'une expression ou un bloc
- Condition est une expression booléenne
- La forme alternative représente une bifurcation entre deux partie du code, selon une condition
 - Forme augmentée par le mot clé `else`, suivi d'une expression ou un bloc
- Les structures `if-else` peuvent s'enchaîner

```
if (reponse >= 0) {  
    System.out.println("La réponse est positive.");  
}  
  
if (reponse >= 0) {  
    System.out.println("La réponse est positive.");  
} else {  
    System.out.println("La réponse est négative.");  
}  
  
if (reponse > 0) {  
    System.out.println("La réponse est positive.");  
} else if (reponse < 0) {  
    System.out.println("La réponse est négative.");  
} else {  
    System.out.println("La réponse est nulle.");  
}
```

Structure de branchement switch

- Permet un branchement du code selon la valeur d'une expression
 - Mot clé `switch` suivi d'une expression, suivi de plusieurs entrées
 - Chaque entrée est identifiée par le mot clé `case` suivi d'une valeur *constante** puis double point (`:`)
 - Une seule entrée peut être identifiée par le mot clé `default` puis double point

```
// Switch avec un chaîne de caractère
switch (lireTemps().toLowerCase()) {
    case "jour":
        System.out.println("Bonjour");
        faireCafe();
        break;
    case "soir":
        System.out.println("Bonsoir");
        break;
    default:
        System.out.println("Salut");
}
```

* <https://docs.oracle.com/javase/specs/jls/se17/html/jls-15.html#jls-15.29>

Structure de branchement switch

- Expression peut être de type
 - primitif discret : `byte`, `short`, `char`, et `int`
 - chaîne de caractère `String`
 - énumération : sous-classe de `Enum`
- Le contrôle ne quitte le bloc qu'avec le mot clé `break` (ou à la fin du bloc)
 - Le contrôle peut s'écouler au travers une clause `case` ou `default` non précédé par `break`

```
switch (jour) {  
    case MONDAY:  
    case TUESDAY:  
        nombreHeure = 7;  
        break;  
    case WEDNESDAY:  
        nombreHeure = 2;  
        break;  
    case THURSDAY:  
        nombreHeure = 6;  
        break;  
    case FRIDAY:  
        nombreHeure = 3;  
        break;  
    default:  
        nombreHeure = 0;  
}
```

Expression switch en Java 14

- Un raccourci pour éviter l'oubli de `break`
 - Avec la syntaxe de flèche `->` qui remplace le double point
 - Une entrée peut être une seule instruction ou un bloc
- Structure `switch` est une expression
 - Peut rendre une valeur
 - En utilisant le mot-clé mot réservé `yield`
 - Pas besoin de `break` dans ce cas
- Possibilité de regrouper plusieurs valeurs dans une même entrée `case`
 - Séparées par des virgules

```
// Syntaxe simplifiée sans break
switch (lireTemps().toLowerCase()) {
    case "jour" -> {
        System.out.println("Bonjour");
        faireCafe();
    }
    case "soir" -> System.out.println("Bonsoir");
    default -> System.out.println("Salut");
}

// Switch utilisé en tant qu'expression
double nombreHeureTravail = switch (jour) {
    case MONDAY, TUESDAY: yield 7;
    case WEDNESDAY: yield 2;
    case THURSDAY: yield 6;
    case FRIDAY: yield 3;
    default: yield 0;
};
```

Structure d'itération `for`

- Permet d'itérer sur une partie du code
- Forme basique
`for (init; test; pas) code`
 - *init* (optionnel) expression d'initialisation qui sera exécutée une seule fois
 - *test* (optionnel) expression booléenne qui conditionne chaque itération
 - *pas* (optionnel) sera exécuté à la fin de chaque itération
 - *code* le code d'une itération, soit une seule instruction ou un bloc

```
for (int i = 0; i < tableau.length; i++) {  
    System.out.println(tableau[i]);  
}
```

```
// for avec seulement la partie expression  
int i = 0;  
for (; i < tableau.length &&  
    !"stop".equals(tableau[i++]); );  
System.out.printf("Arrêt à %d", i);
```

```
// Plusieurs expression dans pas  
for (int j = 0, k = 10; j < k; j++, k--) {  
    System.out.println(j * k);  
}
```

Structure d'itération for

- Le mot clé `break` force la sortie de la structure
- Le mot clé `continue` force la sortie de l'itération

```
// Structure for basique
String[] tableau = tableau();
for (int i = 0; i < tableau.length; i++) {
    // Si valeur nulle, passer
    if (tableau[i] == null) {
        continue;
    }
    // Si valeur stop, quitter
    if ("stop".equals(tableau[i])) {
        break;
    }
    System.out.println(tableau[i]);
}
```

Structure d'itération for

- Forme améliorée

`for (declaration: expression) code`

- *declaration* doit déclarer une variable, soit de type *T*
- *expression* doit être de type `Iterable<T>` ou `T[]`

- La même utilisation de `break` et `continue` s'applique

```
// Structure for améliorée
String[] tableau = tableau();
for (String item: tableau) {
    // Si valeur nulle, passer
    if (item == null) {
        continue;
    }
    // Si valeur stop, quitter
    if ("stop".equals(item)) {
        break;
    }
    System.out.println(item);
}
```


Structure de boucle while

- Forme de structure

`while (condition) code`

- *condition* expression booléenne conditionnant l'entrée dans la boucle
 - *code* le code d'une itération, soit une seule instruction ou un bloc
- La même utilisation de `break` et `continue` s'applique
- Il faut faire particulièrement attention à ne pas créer une boucle infinie

```
// Structure while
int i = 0;
while (i < tableau.length) {
    String item = tableau[i];
    // Si valeur nulle, passer au suivant
    if (item == null) {
        i++;
        continue;
    }
    // Si valeur stop, quitter
    if ("stop".equals(item)) {
        break;
    }
    System.out.println(item);
    i++;
}
```

Structure de boucle do-while

- Forme de structure

do *code* while (*condition*) *code*

- *condition* expression booléenne conditionnant la prochaine entrée dans la boucle
- *code* le code d'une itération, soit une seule instruction ou un bloc

- La différence avec `while` simple c'est que `code` sera exécuté au moins une fois

```
// Structure do-while
int i = 3;
do {
    System.out.printf("Compte à rebours %d\n", i);
} while(i-- != 0);
```

Structure try-catch-finally

- Un code java peut s'exécuter correctement ou avec erreur
 - Dans le deuxième cas, on dit que le code lève un 'jetable'
 - Un jetable est un objet de la classe `Throwable`
- La structure `try` permet de gérer les cas d'erreur selon le type de l'erreur
- La clause `catch` indique le code à exécuter lors d'une erreur
 - Il peut y avoir plusieurs clauses `catch` qui seront vérifiées dans l'ordre
 - On peut inclure plusieurs types dans une seule clause `catch` grâce à la barre verticale (`|`)
- La clause `finally` indique un code qui sera exécuté dans tous les cas
 - Y compris le cas où le code du `try` quitte la méthode en cours avec `return` ou `throw`
- Il doit y avoir au moins une clause `catch` ou `finally`

Structure try-catch-finally : exemple

```
// Structure try simple
try {
    // Code qui risque lever une exception
    ...
} catch (Exception e) {
    // Le code à exécuter en cas d'erreur
    ...
} finally {
    // Le code à exécuter à la fin, dans tous les cas
    ...
}
```

Structure try-catch-finally : exemple

```
// Structure try simple
try {
    // Code qui risque lever une exception
    return faireDuCafe();
} catch (ReservoirEauVideException e) {
    // Au cas où le réservoir est vide
    remplirReservoirEau()
    return faireDuCafe();
} catch (ReservoirGrainsVideException | MachineEnPanneException e) {
    // Au cas où le réservoir à grains est vide ou la machine est en panne
    System.out.println(e.getMessage());
    return faireDuThe();
} finally {
    // Quoi qu'il arrive, éteindre la machine à la fin
    eteindreLaMachine();
}
```

Structure `try` avec ressource

- En plus que la structure `try` classique, cette structure accepte un objet de type `AutoCloseable` appelé ressource
- La ressource est passée entre parenthèse après le mot clé `try`
- La méthode `close()` de la ressource sera appelée automatiquement dans une clause `finally` implicite
 - Cela n'empêche pas une clause `finally` explicite
- Ni la clause `catch` ni la clause `finally` n'est obligatoire dans ce cas
- Plusieurs ressources peuvent être passées
 - Séparées par des point-virgules
 - Les ressource doivent être finales, ou effectivement finales

Structure try avec ressource : exemple

```
// Structure try avec ressource
try (MachineCafe machine = new MachineCafe()) {
    // Code qui risque lever une exception
    return machine.faireDuCafe();
} catch (ReservoirEauVideException e) {
    // Au cas où le réservoir est vide
    return remplirEau();
} catch (ReservoirGrainsVideException | MachineEnPanneException e) {
    // Au cas où le réservoir à grains est vide ou la machine est en panne
    System.out.println(e.getMessage());
    return faireDuThe();
} finally {
    // Pas besoin d'éteindre la machine, elle le sera automatiquement
    faireLaVaisselle();
}
```

Structure try avec ressource

```
// Ni catch ni finally n'est obligatoire avec ressource ressource
try (MachineCafe machine = new MachineCafe()) {
    // Code qui risque lever une exception
    machine.faireDuCafe();
}

// Une référence à une ressource (au lieu d'une nouvelle déclaration) peut être passée depuis Java 9
Theiere theiere = new Theiere();
// Plusieurs ressources peuvent être passées, séparées par des point-virgules
try (
    MachineCafe machine = new MachineCafe();
    theiere) {
    return machine.faireDuCafe();
} catch (ReservoirGrainsVideException | MachineEnPanneException e) {
    // Ca ne serait pas possible d'utiliser la thièrre ici si on a fait la déclaration dans try
    return theiere.faireDuThe();
}
```


Programmation Objet

La classe

- La classe est un concept phare dans la modélisation orienté objet
- En java des nouvelle classes peuvent être créées avec le mot clé `class`
- Chaque classe a une visibilité*
 - Visibilité publique : indiquée par le mot clé `public` avant `class`
 - Visibilité paquetage : indiqué par absence de mot clé avant `class`
- Une classe peut définir des champs et des méthodes
- On peut aussi définir des blocs du code à l'intérieur d'une classe
 - Non statique : initialisation de l'objet
 - Statique : initialisation de la classe

* Les classees imbriquées peuvent avoir une visibilité privée ou protégée aussi

Constructeurs

Plan de construction d'objet

- Un constructeur est une méthode spéciale
- Il est invoqué lors de la création d'un objet
- Il peut y avoir plusieurs pour la même classe

Constructeurs

- Syntaxe similaire de celui des méthodes
 - Il a toujours **le même nom que la classe**
 - Aucun type de retour
 - Il peut être surchargé (plusieurs constructeur)
 - Il peut déclarer la liste d'exceptions qui peuvent être levées
 - Il a une visibilité
- Généralement, un constructeur explicite n'est pas obligatoire
- Si aucun n'est défini, le compilateur crée un sans argument qui fait simplement appel au constructeur sans argument de la classe mère
 - Si au moins un constructeur est défini explicitement, le compilateur n'en crée aucun
- Si la classe mère n'a pas de constructeur sans argument accessible, la création explicite d'au moins un constructeur devient obligatoire

Constructeurs

```
public class AvecConstructeurSansArgument {  
    public AvecConstructeurSansArgument() {  
        super();  
    }  
}  
  
public class PasBesoinDeConstructeur extends AvecConstructeurSansArgument {}  
public class AvecConstructeurAvecArgument {  
    public AvecConstructeurAvecArgument(String s) {  
        super();  
    }  
}  
  
public class BesoinDeConstructeur extends AvecConstructeurAvecArgument {  
    public BesoinDeConstructeur() {  
        super("Hello");  
    }  
}
```

Le mot clé `this`

- Dans une méthode non statique ou un constructeur le mot clé `this` référence l'objet courant (auto-référence)
 - Dans le cas de méthode : instance sur laquelle la méthode a été appelée
 - Dans le cas de constructeur : instance en cours de création
 - Il permet d'accéder aux autres méthodes et champs avec la syntaxe du point
- Dans un constructeur, si suivi par des parenthèses, il sert à faire appel à un autre constructeur
 - Doit être la toute première instruction dans le corps du constructeur (avant Java 25)
 - Des paramètres peuvent être passés entre parenthèses

Le mot clé `super`

- Dans une méthode redéfinie le mot clé `super` sert à accéder la méthode qu'elle masque
 - `super` suivi d'un point, puis le nom de la méthode avec les paramètres
- Dans un constructeur, suivi par des parenthèses, il sert à faire appel au constructeur de la classe mère
 - Doit être la toute première instruction dans le corps du constructeur (avant Java 25)

Chaîne de construction

- Lors de la création d'un objet, les constructeurs de chaque classe dans la chaîne d'héritage de la classe de création doit être appelés dans l'ordre
 - De constructeur de la classe dérivé au constructeur de la classe mère, jusqu'à `Object`
- Pour ce faire, un constructeur doit impérativement faire appel au constructeur de la classe mère, ou à déléguer à un autre constructeur local
 - Si un constructeur ne fait appel ni au constructeur de la classe mère (via `super()`) ni à un autre constructeur locale (via `this()`), le compilateur essaie de faire appel implicite au constructeur sans argument de la classe mère
 - Si la classe mère ne contient pas de constructeur sans argument, un appel explicite à un autre constructeur, local ou de la classe mère, devient obligatoire

Instanciation de l'objet

- L'instanciation d'un objet se fait grâce au mot clé `new` et en faisant appel à un constructeur
- Le mot clé `new` est suivi par le nom de la classe, puis une paire de parenthèse comprenant une liste (potentiellement vide) d'arguments
- Le résultat de cette expression est un objet du type de la classe évoquée

L'interface

- Un type spécial qui ne définit que le comportement de l'objet
 - Alors qu'une classe définit l'état et le comportement
- Une interface peut pas être instanciée
 - Au même titre qu'une classe abstraite
 - Il faut la faire réaliser par une classe
- Les champs d'une interface sont toujours publique, statique et finale
 - Les mots clés `public`, `static` et `final` sont optionnels
- Par défaut, les méthodes d'une interface sont abstraite et publique
- Une méthode peut avoir une réalisation
 - Elle doit être précédée par le mot clé `default`
- Une méthode peut être privé si précédé par le mot clé `private`
 - Elle doit être concrète et avoir un corps

Interface : exemple

```
public interface Tache {  
    int MAX_TRIALS = 10;  
    Result executer() throws Exception;  
    default void executerEtReessayer() throws Exception {  
        this.executerEtReessayer(MAX_TRIALS);  
    }  
    private void executerEtReessayer(int trial) throws Exception {  
        try {  
            this.executer();  
        } catch (Exception e) {  
            if (trial > 0) {  
                this.executerEtReessayer(--trial);  
            } else {  
                throw e;  
            }  
        }  
    }  
}
```

Héritage - spécialisation

- Une classe peut hériter d'une autre classe
 - Elle est appelée une classe dérivée ou une sous-classe
 - La classe dont elle hérite est appelée classe mère ou superclasse
- La classe dérivée spécialise la classe mère
 - La classe mère généralise les classes dérivées
- La superclasse ne peut pas être finale
- La syntaxe est simple : le nom de classe déclarée est suivi par le mot clé `extends` puis le nom de la classe mère
- Une classe ne peut hériter qu'une seule classe
 - Java ne permet pas l'héritage multiple
- Toute classe hérite de la classe `Object`
 - Directement ou indirectement, explicitement ou implicitement

Réalisation

- Une classe ou une interface peut réaliser une autre interface
 - La classe ou interface réalisant hérite les méthodes de l'interface réalisée
 - Pour être concrète, la classe réalisante doit réaliser toutes les méthodes abstraites de l'interface
- Il est permis de réaliser plusieurs interfaces à la fois
- La syntaxe est simple : nom de la classe ou interfaces réalisant, suivi du mot clé `implements`, puis une liste des interfaces réalisées, séparées par des virgules

Réalisation

```
public interface Aromatique {}  
public interface Comestible {}  
public class Plante {}  
public class Plat {}  
  
class Basilic extends Plante implements Comestible, Aromatique {}  
  
class Tiramisu extends Plat implements Comestible {}  
  
class HuileEssentielle implements Aromatique {}
```

Conversion des types objets

- Une conversion ne change pas la nature de l'objet
 - Seulement le type de référence compris par le compilateur
- Conversions élargissantes
 - D'un type à un de ses supertype (classe et interface dans sa chaîne d'héritage)
 - Automatique, ne demande aucune action spécial
- Conversions rétrécissantes^{*}
 - D'une classe à une de ses sous-classe
 - D'une classe à une interface (non réalisée par la classe)
 - D'une interface à une classe non finale
 - D'une interface à une autre interface (non réalisé par la première)

^{*} La liste complète ici : <https://docs.oracle.com/javase/specs/jls/se17/html/jls-5.html#jls-5.1.6.1>

Conversion explicite des types objets

- Les conversions rétrécissantes nécessitent une conversion explicite
 - Le nom du type cible entre deux parenthèses
- Si aucun rapport entre les deux type : erreur de compilation
- Si l'objet n'est pas du type cible lors de l'exécution : erreur de type

`ClassCastException`

- Pensez à vérifier le type d'objet avec `instanceof`

Réalisation d'une méthode

- Une méthode abstraite héritée d'une classe ou interface mère doit être réalisée dans une classe concrète
 - Sinon, la classe doit être déclarée abstraite
- Elle doit avoir la même signature que la méthode abstraite
- Elle doit avoir un corps : une réalisation
- Elle peut spécialiser le type de retour
 - Le type de retour peut être une sous-classe ou sous-interface du type de retour de la classe mère
- Elle peut spécialiser les types d'erreurs déclarés
 - Elle ne peut pas en ajouter

Redéfinition d'une méthode

- Une méthode dans une classe dérivée peut redéfinir une méthode de la classe mère
 - Elle peut utiliser le mot clé `super` pour faire appel à la dernière
- Même règles que pour la réalisation
 - Elle doit avoir la même signature que la méthode de la classe mère
 - Elle doit avoir un corps
 - Elle peut spécialiser le type de retour
 - Elle peut spécialiser les types d'erreurs déclarés
- La réalisation et la redéfinition des méthodes permettent d'avoir un comportement **polymorphique** d'objet

Classe imbriquée

- Une classe peut avoir une ou plusieurs classes imbriquées
- Une classe imbriquée peut avoir une visibilité `public`, `private`, `protected` ou paquetage par défaut
 - Vs. `public` ou paquetage pour 1^e niveau
- Par défaut, une instance de classe imbriquée appartient à une instance de la classe englobante
 - On parle des **classes internes**
- Accéder à l'instance englobante : `NomDeClasseExterne.this`

```
public class ClasseEnglobante {
    int x = 0;
    class ClasseInterne {
        public void methode() {
            // Accès à l'instance externe
            ClasseEnglobante.this.x++;
        }
    }
    public void methode() {
        // i appartient à l'instance englobante
        ClasseInterne i = new ClasseInterne();
    }
}
...
// De l'extérieur, passer par instance externe
var ce = new ClasseEnglobante();
var ci = ce.new ClasseInterne();
```

Classe imbriquée statique

- Une classe imbriquée indépendante de la classe englobante
- Identifiée par le mot clé `static` devant le son nom
- Une instance d'une classe imbriquée statique ne dépendant pas à une instance de la classe englobante

```
public class ClasseEnglobante {  
    class ClasseInterne {}  
    static class ClasseImbriqueeStatic {}  
    public static void methodeStatique() {  
        /* il n'est pas possible de créer une  
         * instance de ClasseInterne ici */  
        var cis = new ClasseImbriqueeStatic();  
    }  
}  
  
...  
  
// Si publique, peut être accéder de l'extérieur  
new ClasseEnglobante.ClasseImbriqueeStatic();
```

Classe anonyme

- Expression permettant de définir et instancier une classe au même temps
 - À partir de classe mère ou interface
- Syntaxe
 - Mot clé `new`
 - Appel au constructeur de la classe mère, ou nom d'interface suivi par deux parenthèses
 - La définition de classe anonyme entre deux accolade
- Puisque c'est une instantiation, une classe anonyme ne peut pas être abstraite

```
/* Une classe anonyme réalisant Runnable est  
 * créée. Un objet de cette classe est instancié  
 * et passé au constructeur de Thread.  
 */  
/  
Runnable anonyme = new Runnable() {  
    @Override  
    public void run() {  
        System.out.println("Hello thread");  
    }  
};  
Thread thread = new Thread(anonyme);  
/* Le thread est ensuite exécuté en parallèle. */  
thread.start();
```

Expression lambda

- Expression permettant d'instancier un type ayant une seule méthode abstraite
 - Une classe ou une interface
- Syntaxe concise pour réaliser la méthode
 - Liste de paramètre entre parenthèses
 - Les types peuvent être omis
 - Parenthèses optionnelles si un seul paramètre
 - Une flèche ->
 - Le code réalisant la méthode
 - Seule instruction, sans `return`
 - Ou un bloc du code

```
/* Lambda peut remplacer la réalisation d'une  
 * interface avec une seule méthode abstraite.  
 */  
new Thread(() -> System.out.println("Hi")).start();  
/* La méthode réalisée peut rendre une valeur,  
 * et prendre des paramètres.  
 */  
List<Integer> chiffres = new ArrayList<>();  
chiffres.sort((x, y) -> x.compareTo(y));  
/* Elle peut aussi avoir un corps */  
chiffres.sort((x, y) -> {  
    if (x == null || y == null) {  
        throw new RuntimeException("Error !");  
    }  
    return x.compareTo(y);  
});
```

Référence de méthode

- Un raccourci pour les expressions lambda faisant appel à une méthode prédéfinie
- Syntaxe simple avec deux double-points
 - Soit une méthode d'objet
 - Soit une méthode de classe (statique)
 - Soit constructeur (mot clé `new`)
- Selon le contexte, elle peut soit prendre des paramètres ou non
- Selon le contexte, elle peut soit renvoyer une valeur ou non

```
// Méthode d'objet sans param, sans retour :  
new Thread(chiffres::clear).start();  
// En lambda :  
new Thread(() -> chiffres.clear()).start();  
  
// Méthode d'objet avec un param et retour  
chiffres.sort(Integer::compareTo); // En lambda :  
chiffres.sort((x, y) -> x.compareTo(y));  
  
// Méthode de classe avec un param et retour  
chiffres.removeIf(Objects::isNull); // En lambda :  
chiffres.removeIf(o -> Objects.isNull(o));  
  
// Constructeur  
chiffres.replaceAll(Integer::new); // En lambda :  
chiffres.replaceAll(v -> new Integer(v));
```

Les Annotations

Des métadonnées sur le code

- À la déclaration pour annoter
 - Un type (classe ou interface)
 - Un paramètre
 - Une variable locale
 - Un champ
 - Une méthode
 - Un constructeur
 - Un paquetage
 - Une autre annotation
- À l'utilisation pour annoter un type

Annotations

- Des interfaces spéciales créées avec `@interface`
- Peut avoir des méthodes abstraites appelées éléments
 - Ils peuvent avoir des valeurs par défaut
- Lors de leurs utilisation, les éléments peuvent être affectés des valeurs constantes
 - Entre parenthèses, liste des paires nom = valeur séparés par virgules
 - Si seul élément, nom `value` peut être omis
 - Parenthèses peuvent être omises si vide
- Un élément peut avoir un des types suivants
 - Types primitifs, `String`, `Class`, énumération ou annotation
 - Un tableau d'un de ces types

```
enum Niveau {  
    Haut, Moyen, Bas,  
}  
  
@interface Confidentiel {  
    Niveau niveau() default Niveau.Moyen;  
    String value() default "";  
}  
  
@Confidentiel("2018-493 du 20 juin 2018")  
public class CompteBancaire {  
    @Confidentiel(  
        niveau = Niveau.Haut,  
        value = "Peut être partagé sous accord"  
    )  
    public double getSolde() { ... }  
}
```

Exemples des annotations prédéfinies

- `@Override`
 - Désigne une méthode redéfinie (masquée)
 - Elle n'est pas obligatoire
 - Erreur de compilation si la méthode ne redéfinit rien
- `@Deprecated`
 - Désigne un élément déprécié (voué à être supprimé dans des versions futures)
- `@SuppressWarnings`
 - Permet de réprimer certains avertissements

```
interface Calculable {
    int calculer();
}
...
class Expression implements Calculable {
    @SuppressWarnings("deprecation")
    @Override
    public int calculer() {
        return CalculCouteux.aNePasUtiliser();
    }
}
...
public class CalculCouteux {
    @Deprecated(forRemoval = true)
    public static int aNePasUtiliser() { ... }
}
```

Exceptions

Traitement et signalement

- Une exception représente un scénario exceptionnel d'exécution
- C'est un objet de la classe `Throwable`^{*}
- Une exception peut être vérifiée ou non-vérifiée

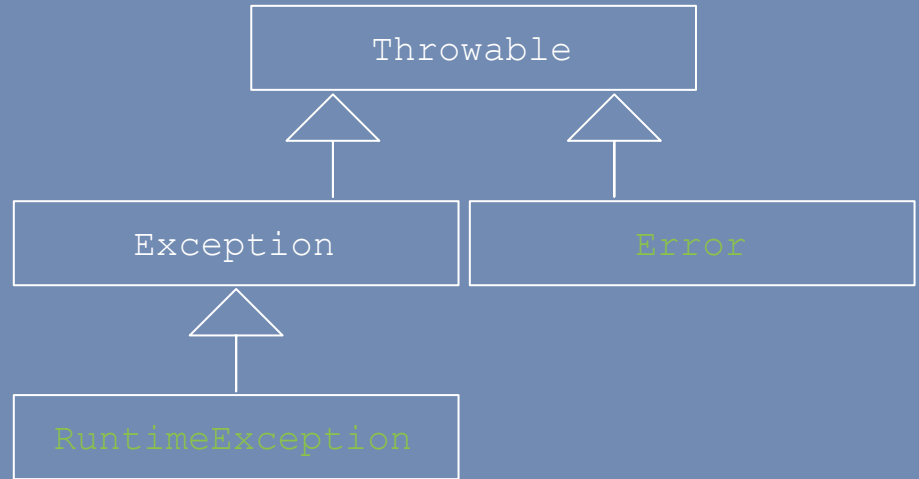
^{*} Appelé précédemment jetable ici

Exceptions vérifiées

- Ce sont des exceptions dont le traitement est vérifié par le compilateur
- Lorsqu'un code peut éventuellement lever une exception vérifiée :
 - L'exception doit être soit traitée
 - Le code doit être entourée par un bloc `try`
 - L'exception doit être explicitement capturée par une clause `catch`
 - Soit signalée
 - La méthode contenant le code signale la possibilité de lever une telle exception avec la phrase `throws` dans son en-tête
- Un code peut éventuellement lever une exception d'un type donné
 - S'il contient une instruction `throw` avec une exception de ce type, ou
 - S'il fait appel à une méthode qui signale ce type dans la clause `throws` de son en-tête

Exceptions non-vérifiées

- Ce sont des exceptions dont le traitement n'est pas vérifié par le compilateur
- Ce sont les exceptions d'un de deux types
 - `Error` : erreur système
 - `RuntimeException` : faille du code
- Aucune obligation, mais elles peuvent être signalées ou capturées et traitées



Génériques

Des types en paramètre

- Permettent aux classes, interfaces et méthodes d'être paramétrées par un ou plusieurs types (classe ou interface)

Génériques

- Avantages des génériques
 - Typage plus fort lors de compilation
 - Éliminations de certaines conversions des types dans le code
 - Permettre d'écrire des algorithmes génériques
 - Algorithme indépendants de type de donnée
- En Java, il existe deux type de générique :
 - Types génériques (classes et interfaces)
 - Méthode générique

Génériques - exemple d'utilisation de List

```
List list = new ArrayList();
list.add("alpha");
list.add("beta");
list.add("gamma");

String alpha = (String) list.get(0);

// Pas d'erreur de compilation
list.add(42);

// Erreur d'exécution
String answer = (String) list.get(3);
```

```
List<String> list = new ArrayList<>();
list.add("alpha");
list.add("beta");
list.add("gamma");

String alpha = list.get(0);

// list.add(42); Ceci serait erreur de compilation
list.add("42");

// Pas besoin de cast, pas d'erreur d'exécution
String answer = list.get(3);
```


Génériques - exemple de création

```
public class ListeChaine {
    private Object valeur;
    private ListeChaine suivant;

    public ListeChaine(Object valeur) {
        this.valeur = valeur;
        this.suivant = null;
    }
    public Object getValeur() {
        return valeur;
    }
}
```

...

```
ListeChaine listeString = new ListeChaine("");
ListeChaine listeInt = new ListeChaine(42);
String s = (String) listeString.getValeur();
```

```
public class ListeChaine<T> {
    private T valeur;
    private ListeChaine<T> suivant;

    public ListeChaine(T valeur) {
        this.valeur = valeur;
        this.suivant = null;
    }
    public T getValeur() {
        return valeur;
    }
}
```

...

```
ListeChaine<String> l = new ListeChaine<>("");
String s = l.getValeur();
```

Les types génériques

- Une classe ou interface qui prend un ou plusieurs types en paramètres
- Les paramètres de type sont fournis après le nom de la classe ou interface
 - Une liste d'identifiants
 - Séparés par des virgules
 - Bornée par des chevrons

```
/** Interface générique */  
interface Conteneur<T> {  
    void set(T value);  
    T get();  
}  
  
/** Classe générique */  
class Boite<T> implements Conteneur<T> {  
    private T value;  
    @Override  
    public void set(T value) {  
        this.value = value;  
    }  
    @Override  
    public T get() {  
        return this.value;  
    }  
}
```

Les types génériques

- Une classe ou interface qui prend un ou plusieurs types en paramètres
- Les paramètres de type sont fournis après le nom de la classe ou interface
 - Une liste d'identifiants
 - Séparés par des virgules
 - Bornée par des chevrons
- Un type générique peut donc être paramétré par plusieurs types

```
/** Classe avec deux paramètres de type */
class Paire<K, V> {
    private K cle;
    private V valeur;

    public Paire() {}

    public Paire(K cle, V valeur) {
        this.cle = cle;
        this.valeur = valeur;
    }

    public K getCle() { return cle; }

    public V getValeur() { return valeur; }
}
```

Les types génériques : invocation

- Une invocation de type générique sert à référencer un tel type dans le code
 - Les paramètres de type sont remplacés par des types (des arguments)
 - Ceci est vrai pour la déclaration et l'instanciation d'un type (après `new`)
- Si les arguments de type peuvent être déduits par le compilateur, ceux-ci peuvent être omis ^{*}
 - Les deux chevrons (diamant) restent obligatoire
 - Ex. lors d'une déclaration

```
Conteneur<Integer> intBox = new Boite<Integer>();

// Arguments de type déduits par la déclaration
Conteneur<String> strBox = new Boite<>();
Paire<Integer, String> paire = new Paire<>();

// Déduits des types d'argument de constructeur
new Paire<>(42, "answer");

// Type brut
Conteneur box = new Boite();

// Un type générique paramétré par un autre
Paire<Integer, Boite<String>> p = new Paire<>();
```

* Depuis Java 7

Les types génériques : type brut

- Une invocation de type générique est aussi appelée **type paramétré**
 - Ex. `Boite<Integer>`
- Un type générique peut être utilisé sans argument de type
 - Pour des raisons de rétro-compatibilité *
 - Il est appelé dans ce cas **type brut**
 - Ex. `Boite`
 - Il est déconseillé d'utiliser les types bruts
- L'argument d'un type paramétré peut lui-même être un type paramétré

```
Conteneur<Integer> intBox = new Boite<Integer>();

// Arguments de type déduits par la déclaration
Conteneur<String> strBox = new Boite<>();
Paire<Integer, String> paire = new Paire<>();

// Déduits des types d'argument de constructeur
new Paire<>(42, "answer");

// Type brut
Conteneur box = new Boite();

// Un type générique paramétré par un autre
Paire<Integer, Boite<String>> p = new Paire<>();
```

* Les génériques ont été introduits en Java 5

Les méthodes génériques

- Une méthode, statique ou non, peut être paramétrée un ou plusieurs types
 - Il s'agit d'une méthode générique
- Les paramètres sont placés avant le type de retour
 - Une liste d'identifiants
 - Séparés par des virgules
 - Bornée par des chevrons
- Lors d'une invocation, les arguments de types peuvent être
 - soit explicites, entre deux chevrons
 - soit implicites, sans besoin de chevrons

```
public <T> void echanger(Boite<T> a, Boite<T> b) {  
    T temp = a.get();  
    a.set(b.get());  
    b.set(temp);  
}  
  
...  
  
/* Invocation avec argument de type explicite */  
this.<Integer>echanger(x, y);  
/* Invocation avec argument de type implicite */  
echanger(x, y);
```

Paramètres de type bornés

```
public <T extends Comparable<T>> T min(T a, T b) {  
    if (a.compareTo(b) > 0) {  
        return b;  
    }  
    return a;  
}  
  
public <T extends Number> double somme(T a, T b) {  
    return a.doubleValue() + b.doubleValue();  
}  
  
public <T extends Number & Comparable<T>> double diff(T a, T b) {  
    if (a.compareTo(b) > 0) {  
        return a.doubleValue() - b.doubleValue();  
    }  
    return b.doubleValue() - a.doubleValue();  
}
```

Paramètres de type bornés

- Un paramètre de type peut être borné par une classe et/ou des interfaces, ex. `Number` **ou** `Comparable`
 - On appelle une telle classe ou interface une **contrainte**
- Ça permet de traiter les objets du type générique borné comme un objet du type de contrainte dans la classe, interface ou méthode générique
- Syntaxe : mot clef `extends` suivi par la liste des contraintes
 - Au plus une classe, doit être déclarée en premier
 - Potentiellement plusieurs interfaces
 - Séparées par des esperluettes (&)

Effacement des types

- Les génériques sont introduits pour améliorer le typage lors de la compilation
- En temps d'exécution, les paramètres des types sont *effacés*
 - Le bytecode ne contient que des classes et interfaces classiques
- Chaque paramètre est remplacé par sa première contrainte s'il est borné, ou `Object` sinon

Effacement des types : exemple

*/** Avant effacement */*

```
class Boite<T> {  
    private T value;  
    public void set(T value) {  
        this.value = value;  
    }  
    public T get() {  
        return this.value;  
    }  
}  
  
public <T extends Number> double somme(T a, T b) {  
    return a.doubleValue() + b.doubleValue();  
}
```

*/** Après effacement, bytecode */*

```
class Boite {  
    private Object value;  
    public void set(Object value) {  
        this.value = value;  
    }  
    public Object get() {  
        return this.value;  
    }  
}  
  
public double somme(Number a, Number b) {  
    return a.doubleValue() + b.doubleValue();  
}
```

Types non réifiâbles

- Une conséquence de l'effacement de type ce sont les types **non réifiâbles**
- Ce sont des types qui ne sont pas disponibles en temps d'exécution
- Il s'agit de paramètres de type dans les types générique
- Interdits dans certains contextes
 - Opérande de `instanceof`
 - Destinataire de `.class`
 - Instanciation des tableaux

```
public <T> void methode(T t) {  
    T[] tab; // Ceci est OK  
    // tab = new T[5]; erreur de compilation  
    Object obj = null;  
    // if (obj instanceof T) {} erreur de compil.  
    // Class c = T.class; erreur de compilation  
    Class c = t.getClass(); // OK  
    T nouveauT = (T) obj; // OK pour la compilation  
}
```

Références

- Oracle Java™ Tutorials

<https://docs.oracle.com/javase/tutorial/java/TOC.html>